St. George's College, Aruvithura

College Road, Aruvithura P.O , Erattupetta, 686122
Affiliated to Mahatma Gandhi University, Kottayam

DEPARTMENT OF COMPUTER APPLICATIONS

BACHELOR OF COMPUTER APPLICATION (BCA)

RECORD OF PRACTICAL WORK

Name : ……

Subject : ……

Semester : ……

St. George's College, Aruvithura

College Road, Aruvithura P.O , Erattupetta, 686122
Affiliated to Mahatma Gandhi University,
Kottayam

DEPARTMENT OF COMPUTER APPLICATIONS

Subject : . . . . . .

Semester : . . . . . .

Certified that this is a bonafide record of practical work done by

NAME : . . . . . .

REG NO : . . . . . . BATCH : . . . . . .

ROLL NO : . . . . . YEAR : . . . . . .

Head of the Department Lecture in-Charge

Submitted for the University Practical Examination held on . . . . . .

Date : . . . . . Internal Examiner External Examiner

INDEX

15	Program to calculate net salary of employees using object array concept.	27/04/2023	37	
16	Implementation of parameterized constructor	27/04/2023	40	
17	Implementation of constructor overloading	27/04/2023	42	
18	Overloading unary operator	27/04/2023	45	
19	Overloading binary operator	27/04/2023	47	
20	Single inheritance by means of private derivation	27/04/2023	49	
21	Single Inheritance by means of public derivation	27/04/2023	51	
22	Multiple Inheritance	27/04/2023	53	
23	Multilevel Inheritance	27/04/2023	55	
24	Hybrid Inheritance	27/04/2023	57	

Program 1

Class Implementation

Aim

Program to implement a class item

Program

```
#include <iostream>
using namespace std;
class item
{
      int number;
      float cost;
      public:
      void getdata(int a,float b);
      void putdata(void)
      {
            cout<<"number:"<<number<<"\n";
            cout<<"cost:"<<cost<<"\n";
      }
};
void item::getdata(int a,float b)
{
      number=a;
      cost=b;
}
int main()
{
      item x;
      cout<<"\nobject x "<<"\n";
      x.getdata(100,299.95);
      x.putdata();
      item y;
      cout<<"\nobject y"<<"\n";
      y.getdata(200,175.50);
      y.putdata();
      return 0;
}
```

Output

```
object x
number:100
cost:299.95

object y
number:200
cost:175.5
```

Program 2

Inline functions

Aim

Program to implement Inline functions

Program

```
# include <iostream>
using namespace std;
     inline float mul(float x,float y)
     {
          return (x*y);
     }
     inline double div(double p, double q)
     {
          return (p/q);
     }
int main()
{
     float a=12.345;
     float b=9.82;
     cout<<mul(a,b)<<"\n";
     cout<<div(a,b)<<"\n";
     return 0;
}
```

Output

```
121.228
1.25713
```

Program 3

Function overloading

Aim

Program to implement Function overloading

Program

```
#include <iostream>
using namespace std;
int volume(int);
double volume(double,int);
long volume(long,int,int);
int main()
{
     cout<<"Calling the volume() function for computing the
volume of a cube-"<<volume(10)<<"\n";
     cout<<"Calling the volume() function for computing the
volume of a cylinder-"<<volume(2.5,8)<<"\n";
     cout<<"Calling the volume() function for computing the
volume of a rectangular box-"<<volume(100L,75,15)<<"\n";
     return 0;
}
int volume(int s)
{
     return (s*s*s);
}
double volume(double r,int h)
{
     return (3.14519*r*r*h);
}
long volume(long l,int b,int h)
{
     return (l*b*h);
}
```

Output

```
Calling the volume() function for computing the volume of a
cube-1000
```

```
Calling the volume() function for computing the volume of a
cylinder-157.26
Calling the volume() function for computing the volume of a
rectangular box-112500
```

Program 4

Default Arguments

Aim

Program to implement Default Arguments

Program

```
#include <iostream>
using namespace std;
int main()
{
	float amount;
	float value(float p,int n,float r=0.15);
	void printline(char ch='*',int len=40);
	printline();
	amount=value(5000.00,5);
	cout<<"\n Final value="<<amount<<"\n\n";
	amount=value(10000.00,5,0.30);
	cout<<"\n Final value="<<amount<<"\n\n";
	printline('=');
	return 0;
}
float value(float p,int n,float r)
{
	int year=1;
	float sum=p;
	while (year<=n)
	{
		sum=sum*(1+r);
		year=year+1;
	}
	return (sum);
}
void printline(char ch,int len)
{
	for(int i=1;i<=len;i++)
	cout<<ch;
```

```
    cout<<"\n";
}
```

Output

```
****************************************
Final value=10056.8

Final value=37129.3

========================================
```

Program 5

Program to implement a class *student* having member functions to compute and display total marks and percentage.

Aim

Program to implement a class student having the following members
Data members – Student name, marks, total, max_mark
Member functions – Assign, compute, display

Program

```
#include <iostream>
using namespace std;
class student
{
      char student_name[25];
      int marks[5];
      float total=0;
      int maximum_mark=500;
      float percentage;
      public:
      void assign();
      void compute();
      void display();
};
void student::assign(void)
{
      int i;
      cout<<"Enter student name:";
      cin>>student_name;
      for(i=0;i<5;i++)
      {
            cout<<"Enter the mark of subject "<<i+1<<":";
            cin>>marks[i];
      }
}
void student::compute(void)
{
      int i;
```

```
      for(i=0;i<5;i++)
      {
            total=total+marks[i];
      }
      percentage=(total/maximum_mark)*100;
}
void student::display(void)
{
      int i;
      cout<<"marks\n";
      for(i=0;i<5;i++)
      {
            cout<<marks[i]<<"\n";
      }
      cout<<"total marks="<<total<<"\n";
      cout<<"percentage="<<percentage<<"\n";
}
int main()
{
      student s;
      s.assign();
      s.compute();
      s.display();
      return 0;
}
```

Output

```
Enter student name:Josekutty
Enter the mark of subject 1:99
Enter the mark of subject 2:89
Enter the mark of subject 3:81
Enter the mark of subject 4:100
Enter the mark of subject 5:95
marks
99
89
```

```
81
100
95
total marks=464
percentage=92.8
```

Program 6

Program to find sum of digits of a number using class

Aim

Program to find sum of digits of a number using class

Program

```
#include <iostream>
using namespace std;
class sum
{
      int num;
      int sum=0;
      int n;
      public:
      void getdata();
      void display();
};
void sum::getdata(void)
{
      int i;
      cout<<"Enter the number:";
      cin>>num;
      for(i=0;num!=0;i++)
      {
            n=num%10;
            sum=sum+n;
            num=num/10;
      }
}
void sum::display(void)
{
      cout<<"sum="<<sum<<"\n";
}
int main()
{
      sum s;
      s.getdata();
```

```
        s.display();
        return 0;
}
```

Output

```
Enter the number:238
sum=13
```

Program 7

Swap values of two variables using *call by value* and *call by reference*

Aim

Program to swap to values of two variables using call by value and call by reference

Program

```
#include <iostream>
using namespace std;
class swap_values
{
    int a,b;
    int temp;
    public:
    void swap_by_value(int,int);
    void swap_by_reference(int *,int *);
};
void swap_values::swap_by_value(int a, int b)
{
    temp=a;
    a=b;
    b=temp;
    cout<<"The swapped values are:\n";
    cout<<a<<"\n"<<b<<"\n";
}
void swap_values::swap_by_reference(int *a,int *b)
{
    temp=*a;
    *a=*b;
    *b=temp;
    cout<<"The swapped values are:\n";
    cout<<*a<<"\n"<<*b<<"\n";
}
int main()
{
    int a,b;
    cout<<"Enter the first number:";
```

```
    cin>>a;
    cout<<"Enter the second number:";
    cin>>b;
    swap_values s;
    s.swap_by_value(a,b);
    s.swap_by_reference(&a,&b);
    return 0;
}
```

Output

```
Enter the first number:100
Enter the second number:50
The swapped values are:
50
100
The swapped values are:
50
100
```

Program 8

Implementation of array within a class

Aim

Program to implement an array within a class

Program

```
#include <iostream>
using namespace std;
class array_imp
{
    int a[10];
    public:
    void setvalue(void);
    void display(void);
};
void array_imp::setvalue(void)
{
    int i;
    cout<<"Enter the numbers:"<<"\n";
    for(i=0;i<10;i++)
    {
        cin>>a[i];
    }
}
void array_imp::display(void)
{
    int i;
    cout<<"The numbers are:"<<"\n";
    for(i=0;i<10;i++)
    {
        cout<<a[i]<<"\n";
    }
}
int main()
{
    array_imp a;
    a.setvalue();
```

```
    a.display();
    return 0;
}
```

Output

```
Enter the numbers:
1
2
3
4
5
6
7
8
9
10
The numbers are:
1
2
3
4
5
6
7
8
9
10
```

Program 9

Implementation of a shopping list of items using a class with arrays as data members for placing an order with a dealer

Aim

Program to create a shopping list of items for which a shop owner can place an order with a dealer every month.The shopping list includes details such as code number and price of each item also perform operations such as adding an item to the list,deleting an item from the list and printing the total value of the order. Implement these operations using a class with arrays as data members.

Program

```
#include <iostream>
using namespace std;
class ITEMS
{
    int itemCode[50];
    float itemPrice[50];
    int count;
    public:
    void CNT(void){count=0;}
    void getitem(void);
    void displaySum(void);
    void remove(void);
    void displayItems(void);
};
void ITEMS::getitem(void)
{
    cout<<"Enter item code:";
    cin>>itemCode[count];
    cout<<"Enter item cost:";
    cin>>itemPrice[count];
    count++;
}
void ITEMS::displaySum(void)
{
    float sum=0;
```

```
    for(int i=0;i<count;i++)
    {
        sum=sum+itemPrice[i];
    }
    cout<<"\nTotal value:"<<sum<<"\n";
}
void ITEMS::remove(void)
{
    int a,i;
    cout<<"Enter item code:";
    cin>>a;
    for(i=0;i<count;i++)
    {
        if(itemCode[i]==a)
        {
            itemPrice[i]=0;
        }
    }
}
void ITEMS::displayItems(void)
{
    cout<<"\nCode  Price\n";
    for(int i=0;i<count;i++)
    {
        cout<<"\n"<<itemCode[i];
        cout<<"  "<<itemPrice[i];
    }
    cout<<"\n";
}
int main()
{
    ITEMS order;
    order.CNT();
    int x;
    do
    {
        cout<<"\nYou can do the following:"
            <<"Enter appropriate number\n";
        cout<<"\n1:Add an item";
```

```
        cout<<"\n2:Display total value";
        cout<<"\n3:Delete an item";
        cout<<"\n4;Display all items";
        cout<<"\n5:Quit";
        cout<<"\n\nWhat is your option?";
        cin>>x;
        switch(x)
        {
            case 1:order.getitem();break;
            case 2:order.displaySum();break;
            case 3:order.remove();break;
            case 4:order.displayItems();break;
            case 5:break;
            default:cout<<"Error in input;try again\n";
        }
    }while(x!=5);
    return 0;
}
```

Output

```
You can do the following:Enter appropriate number

1:Add an item
2:Display total value
3:Delete an item
4;Display all items
5:Quit

What is your option?1
Enter item code:101
Enter item cost:500
You can do the following:Enter appropriate number

1:Add an item
2:Display total value
3:Delete an item
4;Display all items
5:Quit
```

```
What is your option?1
Enter item code:102
Enter item cost:550
You can do the following:Enter appropriate number

1:Add an item
2:Display total value
3:Delete an item
4;Display all items
5:Quit

What is your option?2
Total value:1050

You can do the following:Enter appropriate number

1:Add an item
2:Display total value
3:Delete an item
4;Display all items
5:Quit

What is your option?4
Code  Price

101  500
102  550

You can do the following:Enter appropriate number

1:Add an item
2:Display total value
3:Delete an item
4;Display all items
5:Quit

What is your option?3
Enter item code:101
```

```
You can do the following:Enter appropriate number

1:Add an item
2:Display total value
3:Delete an item
4;Display all items
5:Quit

What is your option?4
Code  Price

101  0
102  550

You can do the following:Enter appropriate number

1:Add an item
2:Display total value
3:Delete an item
4;Display all items
5:Quit

What is your option?5
```

Program 10

Program to find transpose of a matrix using friend function and returning object concept.

Aim

Program to find the transpose of a matrix using friend function

Program

```
#include <iostream>
using namespace std;
class matrix
{
    int m[3][3];
    public:
    void read(void)
    {
        int i,j;
        cout<<"Enter the elements of the matrix:\n";
        for(i=0;i<3;i++)
        {
            for(j=0;j<3;j++)
            {
                cout<<"m["<<i<<"]["<<j<<"]:";
                cin>>m[i][j];
            }
        }
    }
    void display (void)
    {
        int i,j;
        for(i=0;i<3;i++)
        {
            for(j=0;j<3;j++)
            {
                cout<<m[i][j]<<"\t";
            }
            cout<<"\n";
```

```
        }
    }
    friend matrix trans(matrix);
};
matrix trans(matrix m1)
{
    matrix m2;
    int i,j;
    for(i=0;i<3;i++)
    {
        for(j=0;j<3;j++)
        {
            m2.m[i][j]=m1.m[j][i];
        }
    }
    return (m2);
};
int main()
{
    matrix mat1,mat2;
    mat1.read();
    cout<<"\nYou entered the following matrix\n";
    mat1.display();
    mat2=trans(mat1);
    cout<<"\nTransposed matrix:\n";
    mat2.display();
    return 0;
}
```

Output

```
Enter the elements of the matrix:
m[0][0]:2
m[0][1]:4
m[0][2]:6
m[1][0]:8
m[1][1]:1
m[1][2]:3
m[2][0]:5
```

```
m[2][1]:7
m[2][2]:9
You entered the following matrix
2    4    6
8    1    3
5    7    9

Transposed matrix:
2    8    5
4    1    7
6    3    9
```

Program 11

Implementation of linear search using arrays.

Aim

Program to find an element from an array by means of linear search. Implement using class

Program

```
#include <iostream>
using namespace std;
class ls
{
    int a[100];
    public:
    void read(int n)
    {
        int i;
        cout<<"Enter the numbers\n";
        for(i=0;i<n;i++)
        {
            cin>>a[i];
        }
    }
    void linearsearch(int n,int num)
    {
        int i,flag;
        for(i=0;i<n;i++)
        {
            if(a[i]==num)
            {
                flag=1;
            }
        }
        if(flag==1)
        {
            cout<<"The number is present in the array\n";
        }
        else
```

```
        {
            cout<<"The number is not present\n";
        }
    }
};
int main()
{
    int n,num;
    ls l;
    cout<<"Enter the number of numbers:";
    cin>>n;
    l.read(n);
    cout<<"Enter the number to be searched:";
    cin>>num;
    l.linearsearch(n,num);
    return 0;
}
```

Output

```
Enter the number of numbers:4
Enter the numbers
10
25
32
40
Enter the number to be searched:25
The number is present in the array
```

Program 12

Finding the largest and smallest numbers from a list of integers using class

Aim

Program to find the largest and smallest numbers from a list of integers.

Implement using class

Program

```
#include <iostream>
using namespace std;
class largesmall
{
    int a[100];
    public:
    void read(int n)
    {
        int i;
        cout<<"Enter the numbers\n";
        for(i=0;i<n;i++)
        {
            cin>>a[i];
        }
    }
    void largest(int n)
    {
        int large,i=0;
        large=a[i];
        for(i=0;i<n;i++)
        {
            if(a[i]>large)
            {
                large=a[i];
            }
        }
        cout<<"The largest number is "<<large<<"\n";
    }
    void smallest(int n)
    {
```

```
        int small,i=0;
        small=a[i];
        for(i=0;i<n;i++)
        {
            if(a[i]<small)
            {
                small=a[i];
            }
        }
        cout<<"The smallest number is "<<small<<"\n";
    }
};
int main()
{
    int n;
    largesmall x;
    cout<<"Enter the number of numbers:";
    cin>>n;
    x.read(n);
    x.largest(n);
    x.smallest(n);
    return 0;
}
```

Output

```
Enter the number of numbers:4
Enter the numbers
52
28
34
5
The largest number is 52
The smallest number is 5
```

Program 13

Program to exchange the private data values of two classes using a common friend function named exchange.

Aim

Program to exchange the private data values of two classes. Use a common friend function named exchange (friend to both the classes) to do the exchange.

Program

```
#include <iostream>
using namespace std;
class class_2;
class class_1
{
    int value1;
    public:
    void indata(int a)
    {
        value1=a;
    }
    void display(void)
    {
        cout<<value1<<"\n";
    }
    friend void exchange(class_1 &,class_2 &);
};
class class_2
{
    int value2;
    public:
    void indata(int a)
    {
        value2=a;
    }
    void display(void)
    {
        cout<<value2<<"\n";
    }
```

```
    friend void exchange(class_1 &,class_2 &);
};
void exchange(class_1 &x,class_2 &y)
{
    int temp=x.value1;
    x.value1=y.value2;
    y.value2=temp;
}
int main()
{
    class_1 c1;
    class_2 c2;
    c1.indata(100);
    c2.indata(200);
    cout<<"Values before exchange"<<"\n";
    c1.display();
    c2.display();
    exchange(c1,c2);
    cout<<"Values after exchange"<<"\n";
    c1.display();
    c2.display();
    return 0;
}
```

Output

```
Values before exchange
100
200
Values after exchange
200
100
```

Program 14

Implementation of static variables and static function

Aim

Write a CPP program with a static function show_count () which displays the no.of objects created should be maintained by a static variable count

Program

```
#include <iostream>
using namespace std;
class test
{
      int code;
      static int count;
      public:
      void setcode(void)
      {
            code=++count;
      }
      void showcode(void)
      {
            cout<<"object number:"<<code<<"\n";
      }
      static void showcount(void)
      {
            cout<<"count:"<<count<<"\n";
      }
};
int test::count;
int main()
{
      test t1,t2;
      t1.setcode();
      t2.setcode();
      test::showcount();
      test t3;
      t3.setcode();
      test::showcount();
      t1.showcode();
```

```
    t2.showcode();
    t3.showcode();
    return 0;
}
```

Output

```
count:2
count:3
object number:1
object number:2
object number:3
```

Program 15

Program to calculate net salary of employees using object array concept.

Aim

Program to read the data of N employees (using object array) and compute and display net salary of each employee where D.A=17% of the basic, Income Tax=20% of the gross salary

Program

```
#include <iostream>
using namespace std;
class emp
{
     float basic,da,it,netsal,gsal;
     char name[20],num[10];
     public:
     void getdata();
     void net_sal();
     void dispdata();
};
void emp::getdata()
{
     cout<<"\n Enter employee number:";
     cin>>num;
     cout<<"\n Enter employee name:";
     cin>>name;
     cout<<"\n Enter employee basic salary in rs:";
     cin>>basic;
}
void emp::net_sal()
{
     da=((.17)*basic);
     gsal=da+basic;
     it=((.2)*gsal);
     netsal=gsal-it;
}
void emp::dispdata()
{
```

```
        cout<<"\n employee number:"<<num;
        cout<<"\n employee name:"<<name;
        cout<<"\n employee DA:"<<da;
        cout<<"\n employee gross salary:"<<gsal;
        cout<<"\n employee income tax:"<<it;
        cout<<"\n employee net salary:"<<netsal<<"rs\n";
}
int main()
{
        emp ob[5];
        int n;
        cout<<"\n Enter the number of employee:";
        cin>>n;
        for(int i=0;i<n;i++)
        {
                ob[i].getdata();
                ob[i].net_sal();
        }
        cout<<"\n\n Employee Details:";
        for(int i=0;i<n;i++)
        {
                cout<<"\n\n Employee:"<<i+1
                <<"\n.................";
                ob[i].dispdata();
        }
        return 0;
}
```

Output

```
Enter the number of employee:2
Enter employee number:1
Enter employee name:Josekutty
Enter employee basic salary in rs:60000
Enter employee number:2
Enter employee name:Benson
Enter employee basic salary in rs:50000
Employee Details:
```

```
 Employee:1
.................
 employee number:1
 employee name:Josekutty
 employee DA:10200
 employee gross salary:70200
 employee income tax:14040
 employee net salary:56160rs


 Employee:2
.................
 employee number:2
 employee name:Benson
 employee DA:8500
 employee gross salary:58500
 employee income tax:11700
 employee net salary:46800rs
```

Program 16

Implementation of parameterized constructor

Aim

Program to implement a class integer with a parameterized constructor

Program

```
#include <iostream>
using namespace std;
class integer
{
	int m,n;
	public:
	integer(int,int);
	void display(void)
	{
		cout<<"m="<<m<<"\n";
		cout<<"n="<<n<<"\n";
	}
};
integer::integer(int x,int y)
{
	m=x;
	n=y;
}
int main()
{
	integer int1(0,100);
	integer int2=integer(25,75);
	cout<<"\nOBJECT1"<<"\n";
	int1.display();
	cout<<"\nOBJECT2"<<"\n";
	int2.display();
	return 0;
}
```

Output

```
OBJECT1
m=0
n=100

OBJECT2
m=25
n=75
```

Program 17

Implementation of constructor overloading

Aim

Implement constructor overloading by means of a menu driven CPP program – area of circle, triangle and rectangle

Program

```
#include <iostream>
#include <math.h>
#include <stdlib.h>
using namespace std;
class area
{
      float ar;
      public:
      area(float r)
      {
            ar=3.14*r*r;
      }
      area(float a,float b,float c)
      {
            float s;
            s=(a+b+c)/2;
            ar=s*(s-a)*(s-b)*(s-c);
            ar=pow(ar,.5);
      }
      area(float l,float b)
      {
            ar=l*b;
      }
      void display()
      {
            cout<<"\n Area:"<<ar;
      }
};
int main()
{
```

```
	int ch;
	float x,y,z;
	do
	{
		cout<<"\n\n 1.Area of circle";
		cout<<"\n 2.Area of Rectangle";
		cout<<"\n 3.Area of Triangle";
		cout<<"\n 4.exit";
		cout<<"\n\n Enter your choice:";
		cin>>ch;
		switch(ch)
		{
			case 1:
			{
				cout<<"\n Enter the radius of the circle:";
				cin>>x;
				area a1(x);
				a1.display();
			}
				break;
			case 2:
			{
				cout<<"\n Enter the length and breadth of
the rectangle:";
				cin>>x>>y;
				area a2(x,y);
				a2.display();
			}
				break;
			case 3:
			{
				cout<<"\n Enter the sides of the traingle:";
				cin>>x>>y>>z;
				area a3(x,y,z);
				a3.display();
			}
				break;
			case 4:
				exit(0);
```

```
                default:
                     cout<<"\n\ninvalid choice";
                }
     }while(ch!=4);
     return 0;
}
```

Output

```
1.Area of circle
2.Area of Rectangle
3.Area of Triangle
4.exit

Enter your choice:2
Enter the length and breadth of the rectangle:5
8
Area:40

1.Area of circle
2.Area of Rectangle
3.Area of Triangle
4.exit

Enter your choice:3
Enter the sides of the traingle:4
7
5
Area:9.79796

1.Area of circle
2.Area of Rectangle
3.Area of Triangle
4.exit

Enter your choice:4
```

Program 18

Overloading unary operator

Aim

Program to implement Operator overloading Unary minus

Program

```
#include <iostream>
using namespace std;
class space
{
     int x,y,z;
     public:
     void getdata(int a,int b ,int c);
     void display(void);
     void operator-();
};
void space::getdata(int a,int b, int c)
{
     x=a;
     y=b;
     z=c;
}
void space::display(void)
{
     cout<<"x="<<x<<" ";
     cout<<"y="<<y<<" ";
     cout<<"z="<<z<<"\n";
}
void space::operator-()
{
```

```
        x=-x;
        y=-y;
        z=-z;
}
int main()
{
        space s;
        s.getdata(10,-20,30);
        cout<<"s:";
        s.display();
        -s;
        cout<<"-s:";
        s.display();
        return 0;
}
```

Output

```
s:x=10 y=-20 z=30
-s:x=-10 y=20 z=-30
```

Program 19

Overloading binary operator

Aim

Program to implement Operator overloading Binary +

Program

```
#include <iostream>
using namespace std;
class complex
{
     float x,y;
     public:
     complex(){}
     complex(float real,float imag)
     {
          x=real;
          y=imag;
     }
     complex operator +(complex);
     void display(void);
};
complex complex::operator+(complex c)
{
     complex temp;
     temp.x=x+c.x;
     temp.y=y+c.y;
     return (temp);
}
void complex::display(void)
```

```
{
      cout<<x<<"+j"<<y<<"\n";
}
int main()
{
      complex c1,c2,c3;
      c1=complex(2.5,3.5);
      c2=complex(1.6,2.7);
      c3=c1+c2;
      cout<<"c1=";
      c1.display();
      cout<<"c2=";
      c2.display();
      cout<<"c3=";
      c3.display();
      return 0;
}
```

Output

```
c1=2.5+j3.5
c2=1.6+j2.7
c3=4.1+j6.2
```

Program 20

Single inheritance by means of private derivation

Aim

Program to implement Single inheritance by means of Private derivation

Program

```
#include <iostream>
using namespace std;
class B
{
        int a;
        public:
        int b;
        void get_ab();
        int get_a(void);
        void show_a(void);
};
class D:private B
{
        int c;
        public:
        void mul(void);
        void display(void);
};
void B::get_ab(void)
{
        cout<<"Enter the values for a and b:";
        cin>>a>>b;
}
int B::get_a()
{
```

```
        return a;
}
void B::show_a()
{
        cout<<"a="<<a<<"\n";
}
void D::mul()
{
     get_ab();
     c=b*get_a();
}
void D::display()
{
     show_a();
     cout<<"b="<<b<<"\n"<<"c="<<c<<"\n\n";
}
int main()
{
     D d;
     d.mul();
     d.display();
     d.mul();
     d.display();
     return 0;
}
```

Output

```
Enter the values for a and b:4
8
a=4
b=8
c=32

Enter the values for a and b:20
30
a=20
b=30
c=600
```

Program 21

Single Inheritance by means of public derivation

Aim

Program to implement Single Inheritance by means of Public derivation

Program

```
#include <iostream>
using namespace std;
class B
{
      int a;
      public:
      int b;
      void set_ab();
      int get_a(void);
      void show_a(void);
};
class D:public B
{
      int c;
      public:
      void mul(void);
      void display(void);
};
void B::set_ab(void)
{
      a=5;
      b=10;
}
int B::get_a()
{
      return a;
```

```
}
void B::show_a()
{
     cout<<"a="<<a<<"\n";
}
void D::mul()
{
     c=b*get_a();
}
void D::display()
{
     cout<<"a="<<get_a()<<"\n";
     cout<<"b="<<b<<"\n";
     cout<<"c="<<c<<"\n\n";
}
int main()
{
     D d;
     d.set_ab();
     d.mul();
     d.show_a();
     d.display();
     d.b=20;
     d.mul();
     d.display();
     return 0;
}
```

Output

```
a=5
a=5
b=10
c=50

a=5
b=20
c=100
```

Program 22

Multiple Inheritance

Aim

Program to implement Multiple Inheritance

Program

```
#include <iostream>
using namespace std;
class M
{
     protected:
     int m;
     public:
     void get_m(int);
};
class N
{
     protected:
     int n;
     public:
     void get_n(int);
};
class P:public M,public N
{
     public:
     void display(void);
};
void M::get_m(int x)
{
     m=x;
}
void N::get_n(int y)
{
     n=y;
```

```
}
void P::display(void)
{
	cout<<"m="<<m<<"\n";
	cout<<"n="<<n<<"\n";
	cout<<"m*n="<<m*n<<"\n";
}
int main()
{
	P p;
	p.get_m(10);
	p.get_n(20);
	p.display();
	return 0;
}
```

Output

```
m=10
n=20
m*n=200
```

Program 23

Multilevel Inheritance

Aim

Program to implement Multilevel Inheritance

Program

```
#include <iostream>
using namespace std;
class student
{
     protected:
     int roll_number;
     public:
     void get_number(int);
     void put_number(void);
};
void student::get_number(int a)
{
     roll_number=a;
}
void student::put_number()
{
     cout<<"Roll Number:"<<roll_number<<"\n";
}
class test:public student
{
     protected:
     float sub1;
     float sub2;
     public:
     void get_marks(float,float);
     void put_marks(void);
};
void test::get_marks(float x,float y)
```

```
{
     sub1=x;
     sub2=y;
}
void test::put_marks()
{
     cout<<"Marks in sub1="<<sub1<<"\n";
     cout<<"Marks in sub2="<<sub2<<"\n";
}
class result:public test
{
     float total;
     public:
     void display(void);
};
void result::display(void)
{
     total=sub1+sub2;
     put_number();
     put_marks();
     cout<<"Total="<<total<<"\n";
}
int main()
{
     result student1;
     student1.get_number(111);
     student1.get_marks(75.0,59.5);
     student1.display();
     return 0;
}
```

Output

```
Roll Number:111
Marks in sub1=75
Marks in sub2=59.5
Total=134.5
```

Program 24

Hybrid Inheritance

Aim

Program to implement Hybrid Inheritance

Program

```
#include <iostream>
using namespace std;
class student
{
      protected:
      int roll_number;
      public:
      void get_number(int a)
      {
            roll_number=a;
      }
      void put_number(void)
      {
            cout<<"Roll No:"<<roll_number<<"\n";
      }
};
class test:public student
{
      protected:
      float part1,part2;
      public:
      void get_marks(float x,float y)
      {
            part1=x;
            part2=y;
      }
      void put_marks(void)
      {
            cout<<"Marks Obtained:"<<"\n"
```

```
                <<"part1="<<part1<<"\n"<<"part2="<<part2<<"\n";
        }
};
class sports
{
        protected:
        float score;
        public:
        void get_score(float s)
        {
                score=s;
        }
        void put_score(void)
        {
                cout<<"Sports wt:"<<score<<"\n\n";
        }
};
class result:public test,public sports
{
        float total;
        public:
        void display(void);
};
void result::display(void)
{
        total=part1+part2+score;
        put_number();
        put_marks();
        put_score();
        cout<<"Total score:"<<total<<"\n";
}
int main()
{
        result student_1;
        student_1.get_number(1234);
        student_1.get_marks(27.5,33.0);
        student_1.get_score(6.0);
        student_1.display();
        return 0;
```

```
}
```

Output

```
Roll No:1234
Marks Obtained:
part1=27.5
part2=33
Sports wt:6

Total score:66.5
```